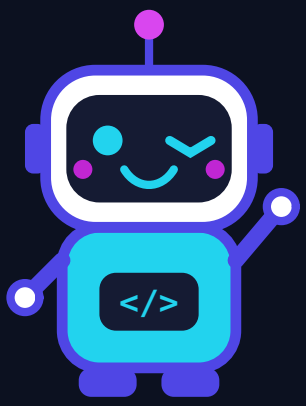
Kod Kahramanları için Programlama Serisi

TürKod ile Kodlama Macerası

Türkçe konuşan bilgisayar dilini birlikte öğreniyoruz!



ilk-programım.trpy

```
1  yazdır("Merhaba, dünya!")
```

>_ terminal

```
Merhaba, dünya!
```

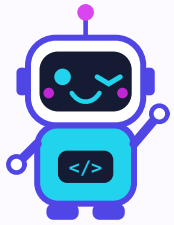
- Renkli, eğlenceli anlatım
- 50+ çalıştırılmış kod örneği
- Büyükler için rehber notlar

Büyüklerle birlikte okumaya ve uygulamalı denemeye uygundur

EK

Merhaba, ben Kodi!

Bu kitabı nasıl okuyacağız?



Merhaba! Ben Kodi, bilgisayarların içinde yaşayan minik bir robotum. Bu kitapta sana bilgisayarla **Türkçe** konuşmayı öğreteceğim. Hazırsan başlayalım!

Kitapta bu kutuları göreceksin

İpucu
Kodi'nin sana küçük bir sırrı var. Kısa yollar ve kolaylıklar burada!

Mini Görev
Kendi başına deneyeceğin bir görev. Bilgisayarında yaz, çalıştır, değiştir!

Dikkat
Dikkat etmen gereken bir şey. Çoğu hata burada gizlenir.

Biliyor muydun?
Merak uyandıran, "vay be!" dedirten bir bilgi.

Bil bakalım
Kafa yorduran bir soru. Cevabı kitabın sonunda kontrol edebilirsin.

Kod kahramanının 3 kuralı

1. **Acele etme.**
Adım adım ilerle. Bir sayfa, bir fikir!

2. **Hata yapmak çok normal!**
Kod kahramanları hep hata yapar, sonra düzeltir. Hata, öğrenmenin kapısıdır.

3. **Her örneği kendin dene.**
Bilgisayarda yazdığın kod, sadece okuduğun koddan on kat daha çok öğretir.

Biliyor muydun?
Bu kitaptaki tüm kod örnekleri gerçekten çalıştırılıp çıktıları kaydedildi. Yani ekranda gördüğün sonuç, kitaptaki sonuçla aynı olmalı. Örnekler **TürKod IDE** içindir (sitedeki Canlı Demo daha sınırlıdır).

EK

İçindekiler

Sertifikan son sayfada seni bekliyor!

Merhaba, ben Kodi! 2

İçindekiler 3

1. Bilgisayar ne yapar? 4

2. Programlama dili nedir? 6

3. TürKod'u kuralım 7

4. TürKod IDE haritası 8

5. İlk programım: yazdır 14

6. Değişkenler: sihirli kutular 16

7. Kutulara ne koyabiliriz? 19

8. Sayılarla matematik 21

9. Yazılarla oynayalım 24

10. Bilgisayar sana soru sorsun 27

11. eğer: karar vermeyi öğren 29

12. döngü: tekrar tekrar yapmak 34

13. Listeler: vagonlu tren 39

14. Fonksiyon: kendi komutunu yap 42

15. Hata avcısı olalım 45

16. Bonus: Kaplumbağa ile çizim 47

17. Mini proje 1: Yaş kahramanı 48

18. Kod kahramanı bilmeceleri 52

19. Kodi'nin sözlüğü 54

20. Büyükler için not 56

Sertifika 57

BÖLÜM 1

Bilgisayar ne yapar?

Bilgisayar çok çalışkan ama kendi başına düşünemez.

Bilgisayar çok hızlıdır. Saniyede milyonlarca iş yapar! Ama bir sorunu var: **ne yapacağını bilmez**. Ona adım adım söylememiz gerekir. Tıpkı bir yemek tarifi gibi.

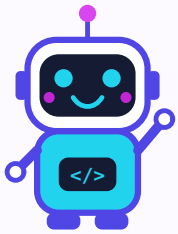


Tarifi yanlış sırayla yaparsan sandviç dağılır! Bilgisayara verdiğimiz talimatlara **kod** denir. Kodun **sırası** çok önemlidir.

> **Biliyor muydun?**
> Bilgisayar kodu yukarıdan aşağıya, satır satır okur ve senin yazdığın sırayla yapar. Ne kadar sabırlı bir arkadaş, değil mi?

Programcı kimdir?

Bilgisayara ne yapacağını yazan insana **programcı** denir. Yazdığı talimatların tamamına da **program** deriz. Oyunlar, uygulamalar, hatta arabanın içindeki ekran bile birer programdır. Sen de az sonra bir programcı olacaksın!



Bilgisayarlar çok akıllı görünür ama aslında çok itaatkârdır. Ne dersen onu yapar, hem de tam olarak dediğin gibi!

Algoritma: adım adım tarif

Bir işi adım adım anlatan tarife **algoritma** denir. Dişini fırçalamak da bir algoritmadır. Aşağıdaki adımlar karışmış. Doğru sırayı bulabilir misin?

Karışık adım	Doğru sıra kaçıncı?
Fırçaya macun sür	1
Fırçayı yıka ve yerine koy	4
Dişlerini iki dakika fırçala	2
Ağzını suyla çalkala	3

Şimdi Kodi'ye **kart oyunu** oynatalım. Kodi'nin bir robot olduğunu unutma; sadece

söylediğin adımları yapar:

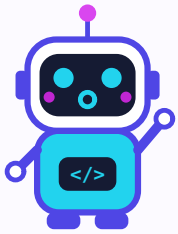
ileri 2 adım → **sağa dön** → **ileri 1 adım** → **dur!**

★ **Mini Görev**

Odada bir yerden başla. Bir arkadaşına ya da büyüğüne bu kartları sırayla oku ve o robot gibi yürüsün. Sonra sırayı değiştir: robot başka bir yere varır mı?

? **Bil bakalım**

Bir sandviçi yapmak için 3 adım gerekiyordu. Peynir ve ekmek adımlarının yerini değiştirirsen ne olur?



Kod yazmak, aslında bilgisayara böyle bir tarif yazmaktır.
Yakında sen de yazacaksın!

BÖLÜM 2

Programlama dili nedir?

Bilgisayarla konuşmanın özel bir yolu.

Biz Türkçe konuşuruz. Bilgisayarlar ise başka bir dil bilir. Bu yüzden **programlama dilleri** icat edildi. TürKod ise özel bir dil: komutları **Türkçe** yazıyoruz. İngilizce bilmene gerek yok!

ilk-programım.trpy

```
1 yazdır("Merhaba, dünya!")
```

>_ TERMİNAL

```
Merhaba, dünya!
```

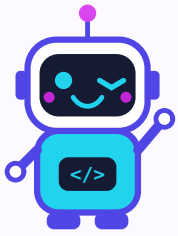
Ne demek istiyoruz?	TürKod'da nasıl yazılır?
Ekrana yaz	`yazdır`
Bilgisayar soru sorsun	`girdi_al`
Eğer öyleyse...	`eğer`
Değilse...	`değilse`
Tekrar tekrar yap	`döngü`
Kendi komutumu yapayım	`fonksiyon`
Bir liste tutayım	`liste = [ ]`

★ **İpucu**

Bu komutları ezberlemene gerek yok. Kitabın sonunda hepsi sözlük sayfasında toplanmış durumda.

i **Biliyor muydun?**

TürKod, yazdığın Türkçe komutları arka planda **Python** adlı dünyaca ünlü bir programlama diline çevirir. Bu yüzden ileride Python öğrenmek çok kolay olacak. Bir hata olursa TürKod sana Türkçe bir mesaj gösterir; onu okumayı Hata Avcısı bölümünde öğreneceğiz.



Sen Türkçe yaz, ben bilgisayarın anladığı dile çeviririm. Anlaştık mı?

BÖLÜM 3

TürKod'u kuralım

Bilgisayarında ilk maceraya çıkma zamanı!

TürKod IDE, **Windows 10 veya 11** yüklü **64 bit** bilgisayarlarda çalışır. Kurulum için bir büyüğünden yardım al.

1. **Bilgisayarını kontrol et**
   Ayarlar → Sistem → Hakkında yolunu izle. "Sistem türü" satırında **64-bit işletim sistemi, x64 tabanlı işlemci** yazıyorsa hazırsın.

2. **Resmî siteye git**
   Büyüğünle şu adrese gir: **yusufx-sys.github.io/turkod-site**. Menüden **Kurulum** ya da **Sürüm Geçmişi** sayfasını aç.

3. **En yeni sürümü indir**
   Sitedeki bağlantı seni **Releases** (sürümler) sayfasına götürür. En üstteki sürümü indir.

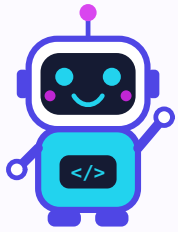
4. **Dosyayı aç**
   İnen **TurKod-Setup-X.Y.Z.exe** kurulum dosyasına çift tıkla ve adımları izle.

5. **TürKod açılıyor!**
   Koyu renkli, renkli yazılı bir ekran görünce hazırsın. Sonraki bölümde onu tanıyacağız.

Dikkat
TürKod'u **sadece resmî siteden** indir. Başka sitelerdeki dosyalar güvenli olmayabilir. Windows "bilinmeyen yayıncı" gibi bir uyarı gösterirse devam etmeden önce yanındaki büyüğüne sor.

Biliyor muydun?
TürKod'un sürümleri RSA denen bir dijital imzayla korunur. Bu, dosyanın yolda değiştirilmediğini kanıtlayan bir tür mühürdür.

İpucu
Sitedeki **Canlı Demo** sayfası kurulum yapmadan denemek içindir ama sınırlıdır: bu kitaptaki döngü, liste ve f"..." örnekleri için kurulu IDE'yi kullan.

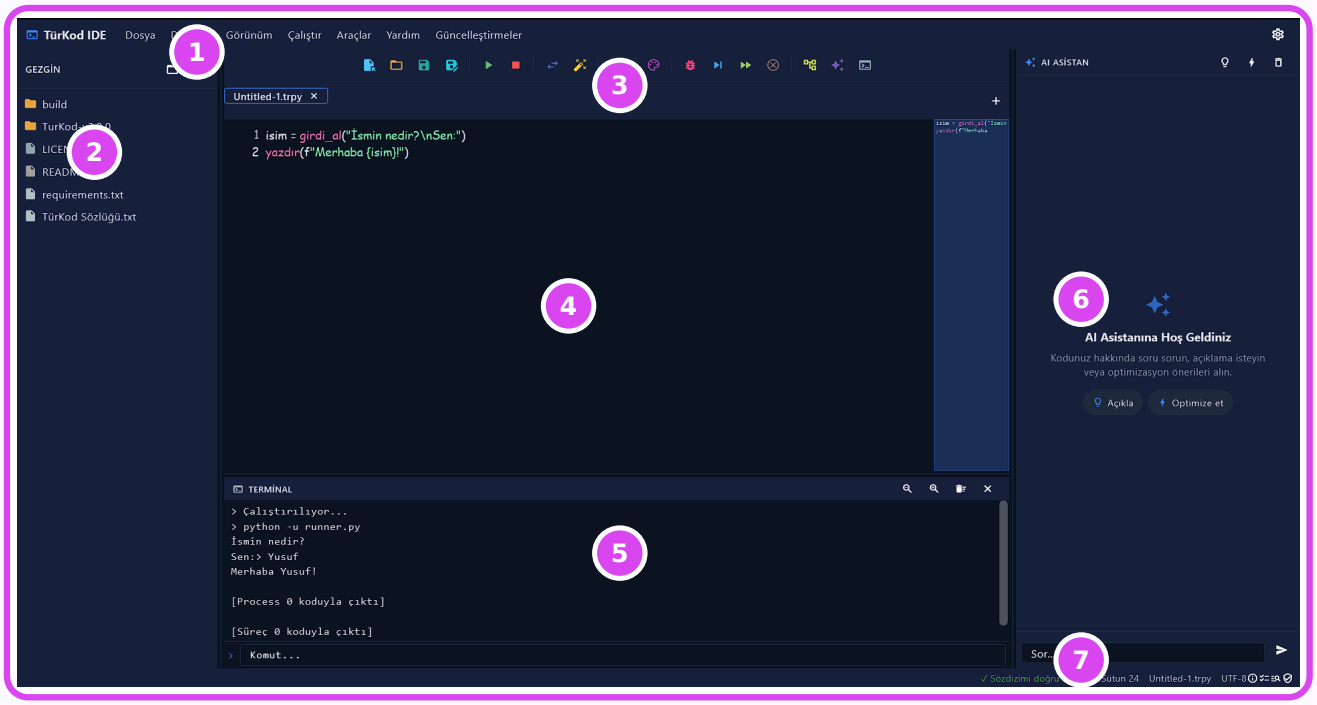


Kurulum bitti mi? Süper! Şimdi TürKod'un içini gezelim.

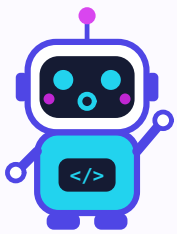
BÖLÜM 4

TürKod IDE haritası

IDE, kod yazdığın renkli çalışma odasıdır. Haydi 7 bölgesini tanıyalım!



No	Bölge	Ne işe yarar?
1	**Menü çubuğu**	Dosya, Düzenle, Yardım... komut listeleri
2	**Gezgin**	Dosyalarının bulunduğu dolap
3	**Araç çubuğu**	Sihirli düğmeler
4	**Editör**	Kod yazdığın defter
5	**Terminal**	Programın konuştuğu yer
6	**AI Asistan**	Yardımcı robot arkadaş
7	**Durum çubuğu**	Küçük bilgi şeridi



Parmağınla numaraları sırayla göster ve her bölgenin adını yüksek sesle söyle. Şimdi tek tek gezeceğiz!

Bil bakalım

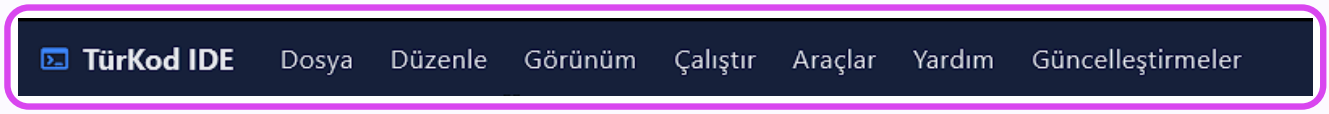
Kodlarını yazdığın yer **hangi numara**? Yeşil **Çalıştır** okunu hangi numarada bulursun?

BÖLÜM 4

Menü çubuğu ve Gezgin

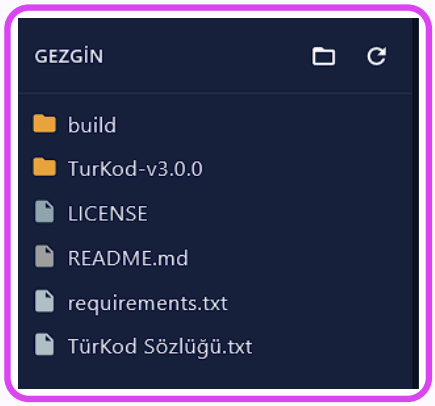
Komutlar yukarıda, dosyalar solda.

1 Menü çubuğu



En üstteki yazılar birer **menü**dür: Dosya, Düzenle, Görünüm, Çalıştır, Araçlar, Yardım ve Güncelleştirmeler. Birine tıklarsan altından bir liste açılır. Sağ üstteki **dişli** ise ayarlar içindir.

2 Gezgin



Solda **GEZGİN** yazan bir panel var. Burası senin **oyuncak dolabın**: yazdığın kodlar dosya olarak burada durur.

- **Klasör simgesi** yeni bir klasör açar.
- **Yenile simgesi** listeyi tazeler.
- Bir dosyaya tıklarsan editörde açılır.

Biliyor muydun?
TürKod dosyalarının adı **.trpy** ile biter. Örneğin: ilk-programım.trpy

Mini Görev
Menülerin üzerine tek tek tıkla ve içlerinde neler olduğuna bak. Hiçbir şeyi bozmazsın, merak etme!



Ne işe yaradığını bilmediğin bir seçenek görürsen, tıklamadan önce büyüğüne sor. Sonra birlikte keşfedin!

BÖLÜM 4

Araç çubuğu: sihirli düğmeler

Menülere girmeden, tek tıkla iş yapmak için.



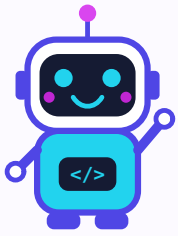
No	Düğme	Ne yapar?
1	**Yeni dosya**	Boş bir sayfa açar. Yeni bir program için!
2	**Aç**	Daha önce yazdığın bir dosyayı getirir.
3	**Kaydet**	Yazdıklarını bilgisayarda saklar.
4	**Farklı kaydet**	Aynı kodu başka bir adla saklar.
5	**Çalıştır** (yeşil ok)	Kodunu başlatır! En çok basacağın düğme.
6	**Durdur** (kırmızı kare)	Çalışan programı durdurur.

★ **İpucu**

Önce **kaydet**, sonra **çalıştır**. Böylece emeğin asla kaybolmaz!

★ **Mini Görev**

Çubukta daha birçok simge var. Fareyi üzerlerine götür ve ne işe yaradıklarını büyüklerinle birlikte keşfet!

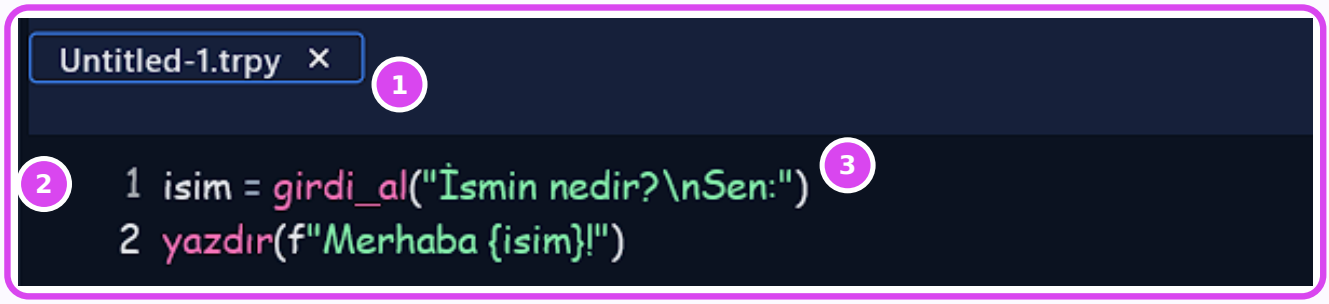


Yeşil ok, oyunlardaki **BAŞLA** düğmesi gibidir. Kırmızı kare ise **DUR** düğmesidir!

BÖLÜM 4

Editör: kod defterin

Kodlarını ortadaki büyük alana yazarsın.



1 **Sekme**
Açık dosyaların adı. **Untitled-1.trpy** henüz kaydedilmemiş yeni bir dosyadır.

2 **Satır numaraları**
Her satırın bir numarası var. Hata olursa "hangi satır?" diye bakıp bulursun.

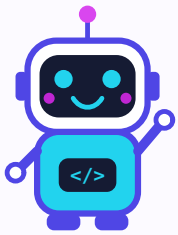
3 **Renkli yazı**
Komutlar ve yazılar farklı renklerle boyanır. Bir şeyi yanlış yazarsan rengi farklı görünebilir!



Minimap

Sağ kenardaki minik kutu, kodunun **kuş bakışı** resmidir. Kodun uzayınca oradan istediğin yere hızlıca gidebilirsin.

> **Biliyor muydun?**
> Editörün üstündeki **+** işareti yeni bir sekme açar. Aynı anda birkaç program yazabilirsin!

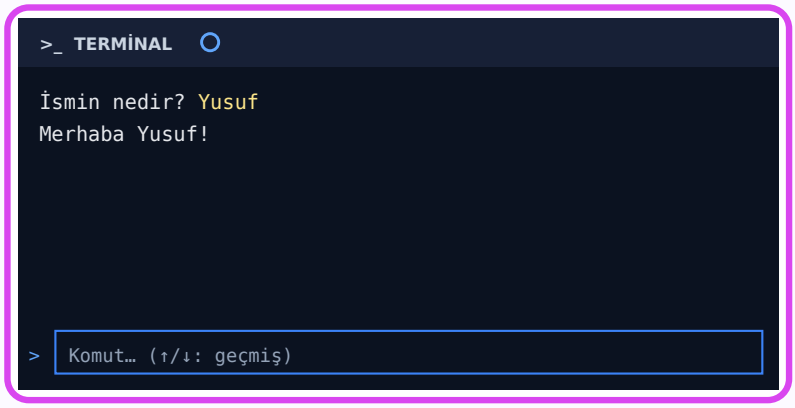


Kodları küçük parçalar halinde yaz ve sık sık **çalıştır**. Böylece hatayı hemen bulursun!

BÖLÜM 4

Terminal ve durum çubuğu

Programın sana cevap verdiği yer.



Terminal

Kodunu çalıştırınca **burası açılır**. Program bir şey yazarsa burada görürsün.

Terminalin sağ üstündeki düğmeler:



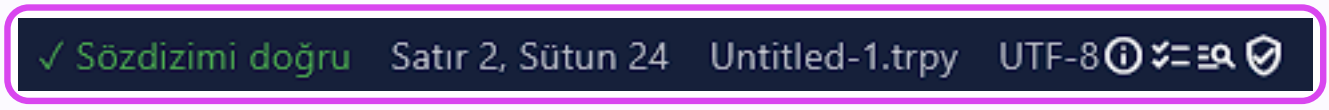
Büyüteçler yazıyı küçültür / büyütür, çöp kutusu ekranı temizler, çarpı paneli kapatır.

1. **Dönen simge**
   TERMİNAL yazısının yanında dönüyorsa program çalışıyor.

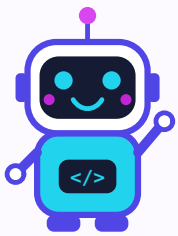
2. **Soru ve cevap**
   Program soru sorarsa cevabı buraya yazıp **Enter**'a basarsın.

3. **[Program 1 koduyla sonlandı]**
   Program hatayla bitti. Sorunsuz biterse bu satır hiç çıkmaz.

Durum çubuğu



En alttaki ince şerit sana **satır ve sütun** numaranı (imlecin nerede?), açık **dosyanın adını** ve yazı kodlamasını (UTF-8) söyler. Yanındaki **"Sözdizimi doğru"** yazısı kodunu kontrol eder.

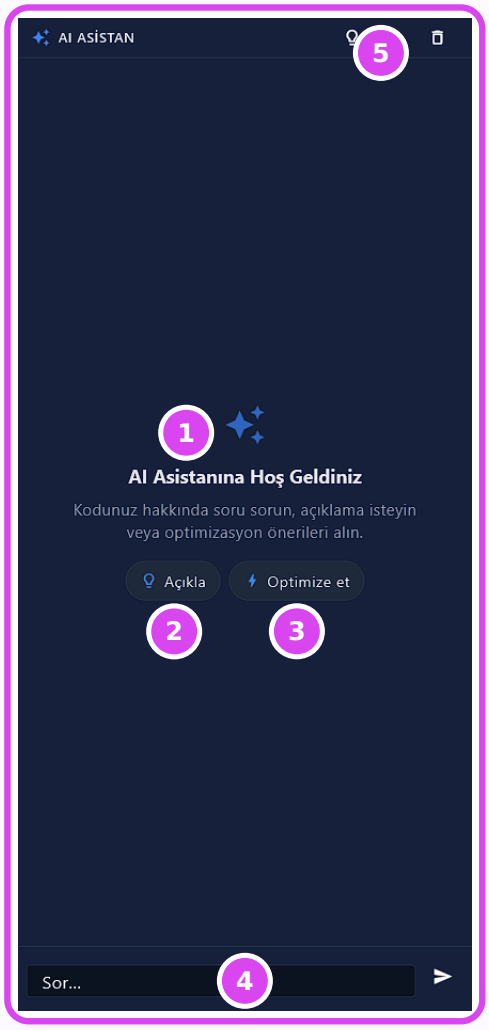


Terminal, programın **konuştuğu** yer. Sen yazarsın, o cevap verir!

BÖLÜM 4

AI Asistan: yardımcı arkadaş

Sağdaki panelde sana yardım eden bir yapay zekâ var.



Kodunu yazarken bir yerde takılırsan **AI Asistan**'a danışabilirsin. Kod hakkında sorulara cevap verir, kodunu açıklar.

1. **Hoş geldin yazısı**
   Panelin boş halidir. Konuşmaya başlayınca sohbet burada görünür.

2. **Açıkla**
   Kodunu sade bir dille anlatmasını ister.

3. **Optimize et**
   Kodunu daha derli toplu yapmak için öneri ister.

4. **Sor... kutusu**
   Kendi sorunu yazarsın. Yanındaki ok simgesiyle gönderirsin.

5. **Üstteki simgeler**
   Kısa yollar ve sohbeti silen bir çöp kutusu var.

> **Dikkat**
> Yapay zekâ bazen **yanılır**. Verdiği cevabı büyüğünle birlikte kontrol et. Önce kendin dene, sonra sor!

BÖLÜM 5

İlk programım: yazdır

Bilgisayara "ekrana bir şey yaz" diyelim!

1. **Yeni dosya aç**
   Araç çubuğunda **Yeni dosya** düğmesine bas.

2. **Kodu yaz**
   Editöre aşağıdaki satırı yaz.

3. **Çalıştır**
   Yeşil **Çalıştır** okuna bas.

4. **Sonucu gör**
   Terminalde çıkan yazıyı oku!

ilk-programım.trpy

```
1 yazdır("Merhaba Dünya!")
```

>_ TERMİNAL

```
Merhaba Dünya!
```

Bu satır nelerden oluşuyor?

Parça	Adı	Ne işe yarar?
`yazdır`	komut	Ekrana yazı yazar
`(`	parantez	Komutun bilgisini içine koyacağımız kapıyı açar
`"Merhaba Dünya!"`	tırnak içinde yazı	Ekrana çıkacak metin
`)`	parantez	Kapıyı kapatır

★ **İpucu**
Bilgisayara yazı göstereceksen mutlaka **tırnak** ("...") içine al. Tırnağı açtıysan kapatmayı unutma!

! **Dikkat**
Komutları küçük harflerle yaz. **yazdır** ile **YAZDIR** aynı şey değildir; bilgisayar ikincisini tanımaz. Tam olarak **yazdır** yaz.

BÖLÜM 5

Birden fazla satır

Bilgisayar satırları yukarıdan aşağıya okur.

Her **yazdır** satırı ekrana bir satır yazı yazar. Üç satır yazarsak üç satır çıkar.

kodi.trpy

```
1 yazdır("Merhaba!")
2 yazdır("Ben Kodi, bir robotum.")
3 yazdır("Seninle tanıştığıma sevindim!")
```

>_ TERMİNAL

```
Merhaba!
Ben Kodi, bir robotum.
Seninle tanıştığıma sevindim!
```



Satırların sırasını değiştirirsen çıktının sırası da değişir. Bilgisayar hep yazdığın sırayla okur!

Yorum satırı: bilgisayara görünmez not

Bir satırın başına **#** (diyez) koyarsan bilgisayar o satırı **görmezden gelir**. Bu, kendine ya da arkadaşına not bırakmak içindir.

yorum.trpy

```
1 # Bu satırı bilgisayar okumaz
2 yazdır("Bu satırı okur!")
```

>_ TERMİNAL

```
Bu satırı okur!
```

Alt satıra geçmek: \n

altsatir.trpy

```
1 yazdır("Birinci satır\nİkinci satır")
```

>_ TERMİNAL

```
Birinci satır
İkinci satır
```

★ **Mini Görev**

Kendi programını yaz: önce adını, sonra en sevdiğin rengi, sonra en sevdiğin hayvanı **yazdır** ile ekrana yazdır. Çalıştır ve gör!

BÖLÜM 6

Değişkenler: sihirli kutular

Bilgisayarın hafızası kutulardan oluşur.

Oyuncaklarını kutularda saklarsın, değil mi? Bilgisayar da bilgileri kutularda saklar. Bu kutulara **değişken** denir. Her kutunun bir **adı** ve içinde bir **değeri** vardır.



kutular.trpy

```
1 isim = "Deniz"
2 yaş = 7
3 yazdır(isim)
4 yazdır(yaş)
```

```
>_ TERMİNAL
Deniz
7
```

`isim = "Deniz"` demek: **"isim adlı kutuya Deniz yaz"**. Eşittir (=) işareti bir **koyma** işaretidir. Sonra `yazdır(isim)` dersen bilgisayar kutuyu açar ve içindekini gösterir.

★ **Mini Görev**

Üç kutu yap: **ad**, **sınıf_no**, **hayvan**. İçlerine kendinle ilgili bilgileri koy ve hepsini yazdır!

★ **İpucu**

Kutunun içinde ne olduğunu unuttuysan **yazdır** ile açıp bakabilirsin. Bilgisayar seni yargılamaz!

BÖLÜM 6

Kutunun içini değiştirmek

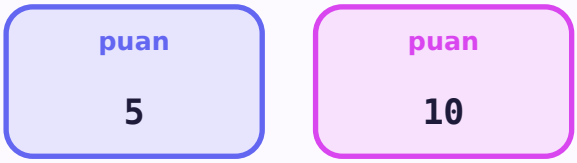
Değişken adı "değişken" çünkü içi değişebilir!

değiştir.trpy

```
1 puan = 5
2 yazdır(puan)
3 puan = 10
4 yazdır(puan)
```

>_ TERMİNAL

```
5
10
```



İkinci satırdan sonra kutunun içindeki **5** silinir, yerine **10** gelir. Eski değer geri gelmez!

Kutuyla hesap yapmak

artır.trpy

```
1 puan = 5
2 puan = puan + 1
3 yazdır(puan)
4 puan += 4
5 yazdır(puan)
```

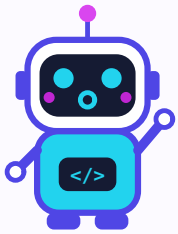
>_ TERMİNAL

```
6
10
```

puan = puan + 1 demek: "kutudaki sayıya 1 ekle ve yine aynı kutuya koy". **puan += 4** de aynı işi kısa yazar: "4 ekle".

Bil bakalım

x = 3 yazdın. Sonra **x = x + 2** yazdın. Şimdi x'in içinde kaç var?



İçi değişen kutular sayesinde oyunlarda puanı artırırız, canı azaltırız. Hepsi değişkenle olur!

BÖLÜM 6

Kutulara isim verme kuralları

Kutunun adı seçerken uyman gerekenler.

1. **Harf, rakam ve alt çizgi (_) kullanabilirsin.**
   ç, ğ, ı, ö, ş, ü harfleri de olur: **yaş**, **güneş**.

2. **İsim rakamla başlayamaz.**
   1sayi olmaz, **sayi1** olur.

3. **İsimde boşluk olamaz.**
   benim yaşım olmaz, **benim_yaşım** olur.

4. **Büyük ve küçük harf farklıdır.**
   Puan ile **puan** iki ayrı kutudur.

birleştir.trpy

```
1 isim = "Deniz"
2 yaş = 7
3 yazdır(f"Ben {isim}, {yaş} yaşındayım.")
```

>_ TERMİNAL

```
Ben Deniz, 7 yaşındayım.
```

Yazıların içine kutu koymanın sihirli yolu: tırnağın önüne **f** yaz, kutunun adını **{ }** içine koy. Bilgisayar kutunun içindekini oraya yerleştirir. Buna **f-yazı** diyeceğiz.

TürKod'un zaten bildiği kelimeler

Bazı kelimeler TürKod'da özel bir komuttur. Bunları kutu adı yapmaktan kaçın, yoksa karışıklık olabilir:

Kelime	Ne yapar?	Kelime	Ne yapar?
`toplam`	Sayıları toplar	`tip`	Türü söyler
`say`	Kaç tane var? diye sayar	`liste`	Liste yapar
`uzunluk`	Uzunluğu ölçer	`metin`	Yazıya çevirir
`aralık`	Sayı yolu çizer	`sırala`	Sıraya dizer

★ **İpucu**
Kutu adını görevini anlatacak şekilde seç: **puan**, **can**, **oyuncu_adı**. Kısa ve anlamlı isimler kodunu okunur yapar.

! **Dikkat**
Kutuyu hiç oluşturmadan yazdırmaya çalışma. Önce **isim = ...** ile oluştur, sonra kullan.

BÖLÜM 7

Kutulara ne koyabiliriz?

Dört önemli tür var. Hepsini tanıyalım!

Kutunun içinde farklı türde şeyler olabilir. Tıpkı bir oyuncak kutusunda hem araba hem bebek bulunabildiği gibi.

Tür	Ne demek?	Örnek	Nasıl tanırsın?
metin	Harflerden oluşan yazı	`"elma"`	Tırnak içindedir
sayı	Hesap yapabileceğin rakamlar	`7` veya `3.5`	Tırnak yoktur
mantıksal	Sadece iki değeri var	`Doğru` / `Yanlış`	Işık düğmesi gibi
liste	Birçok şeyi yan yana tutar	`[1, 2, 3]`	Köşeli parantez

Bir şeyin türünü bilgisayara sor: tip()

tür-sor.trpy

```
yazdır(tip("elma"))
yazdır(tip(7))
yazdır(tip(3.5))
yazdır(tip(Doğru))
yazdır(tip([1, 2, 3]))
```

>_ TERMİNAL

```
<class 'str'>
<class 'int'>
<class 'float'>
<class 'bool'>
<class 'list'>
```

Bilgisayar türleri **kendi dilinde** söyler. Türkçesi şöyle:

Ekranda görünen	Türkçesi
`str`	metin (yazı)
`int`	tamsayı
`float`	ondalıklı sayı (virgüllü)
`bool`	mantıksal (Doğru / Yanlış)
`list`	liste

Biliyor muydun?
Bilgisayarda ondalıklı sayılarda virgül yerine **nokta** kullanırız: 3,5 değil **3.5** yazılır.

BÖLÜM 7

Türleri çevirmek

Bir kutunun içindekini başka türe dönüştürebilirsin.

Bazen elimizdeki yazıyı sayıya ya da sayıyı yazıya çevirmemiz gerekir. Bunun için üç sihirli komut var:

Komut	Ne yapar?	Örnek	Sonuç
`tamsayı()`	Sayıya çevirir	`tamsayı("42")`	42
`ondalıklı()`	Virgüllü sayıya çevirir	`ondalıklı("3.14")`	3.14
`metin()`	Yazıya çevirir	`metin(100)`	"100"

dönüştür.trpy

```
a = "7"
b = tamsayı(a) + 1
yazdır(b)
yazdır(tip(a))
yazdır(tip(b))
```

```
>_ TERMİNAL
8
<class 'str'>
<class 'int'>
```

Yazı ile sayı yan yana gelemez

Yazıyı ve sayıyı **+** ile yapıştırmaya çalışırsan bilgisayar şaşırır. İki yolu var:

yapıştır.trpy

```
yaş = 7
yazdır("Yaşım: " + metin(yaş))
yazdır(f"Yaşım: {yaş}")
```

```
>_ TERMİNAL
Yaşım: 7
Yaşım: 7
```

Birinci yolda sayıyı `metin()` ile yazıya çevirdik. İkinci yol olan **f-yazı** bunu bizim yerimize yapar. Çoğu zaman f-yazı daha kolaydır!

Hata avı

Şunu yazarsan hata alırsın: **yazdır("Yaşım: " + yaş)**. Yazı ile sayıyı doğrudan toplayamazsın. Hata bölümünde bunun mesajını da göreceğiz.

Mini Görev

Kendi yaşını bir kutuya koy. Sonra **f-yazı** ile "Ben ... yaşındayım" cümlesini yazdır.

BÖLÜM 8

Sayılarla matematik

Bilgisayar harika bir hesap makinesidir!

İşaret	Anlamı	Örnek	Sonuç
+	topla	6 + 4	10
-	çıkar	6 - 4	2
*	çarp	6 * 4	24
/	böl	8 / 2	4.0
//	tam böl	7 // 2	3
%	kalan	7 % 2	1
**	kuvvet	2 ** 3	8

hesap.trpy

```
yazdır(6 + 4)
yazdır(6 - 4)
yazdır(6 * 4)
yazdır(8 / 2)
yazdır(7 // 2)
yazdır(7 % 2)
yazdır(2 ** 3)
```

>_ TERMİNAL

```
10
2
24
4.0
3
1
8
```

Pizza örneği: 7 dilim pizzayı 2 kişiye paylaştırıyoruz. Herkese **3** dilim düşer (**//**) ve **1** dilim artar (**%**).



Biliyor muydun?
/ her zaman virgüllü sonuç verir: 8 / 2 sonucu **4.0** olur. Tam sayı istersen **//** kullan.

İpucu
% çift ve tek sayıları bulmak için çok işe yarar: **sayı % 2** sıfırsa sayı çifttir!

Bil bakalım
yazdır(10 % 3) ne yazar? 10 içinde 3 üç kere var (9 eder). Geriye ne kalır?

BÖLÜM 8

İşlem sırası ve daha fazlası

Bilgisayar da bizim gibi önce çarpar, sonra toplar.

sıra.trpy

```
yazdır(2 + 3 * 4)
yazdır((2 + 3) * 4)
```

>_ TERMİNAL

```
14
20
```

Çarpma ve bölme, toplama ve çıkarmadan **önce** yapılır. Önce toplamak istersen **parantez** kullan.

Ondalıklı sayılar

ondalık.trpy

```
boy = 1.25
yazdır(boy * 2)
yazdır(yuvarla(3.7))
yazdır(yuvarla(3.14159, 2))
yazdır(mutlak(-7))
```

>_ TERMİNAL

```
2.5
4
3.14
7
```

Komut	Ne yapar?
`yuvarla(x)`	En yakın tam sayıya yuvarlar
`yuvarla(x, 2)`	Virgülden sonra 2 basamak bırakır
`mutlak(x)`	Eksi işaretini siler: -7 → 7

Matematik kütüphanesi

Bazı komutlar kutuda hazır bekler. Onları çağırmak için önce **içe_aktar** yazarız.

kütüphane.trpy

```
1 içe_aktar matematik
2
3 yazdır(matematik.karekök(16))
4 yazdır(yuvarla(matematik.pi_sayısı, 2))
```

>_ TERMİNAL

```
4.0
3.14
```

Biliyor muydun?

Karekök, kendisiyle çarpılınca o sayıyı veren sayıdır. 4 x 4 = 16 olduğu için 16'nın karekökü **4**'tür. **Pi sayısı** ise bir dairenin çevresinin çapına oranıdır.

Mini Görev

Kenarı 5 olan bir karenin alanını **5 * 5** ile bulup yazdır. Sonra kenarı 12 yap ve tekrar çalıştır!

BÖLÜM 9

Yazılarla oynayalım

Metinlerin de sihirli komutları var.

Metinleri **nokta (.)** ve komut adıyla değiştirebiliriz. Komutların sonundaki **( )** parantezini unutma!

yazi-oyunu.trpy

```
ad = "deniz"
yazdır(ad.büyük_harf())
yazdır(ad.baş_harf_büyük())
yazdır(uzunluk(ad))
yazdır("MERHABA".küçük_harf())
```

>_ TERMİNAL

```
DENIZ
Deniz
5
merhaba
```

Komut	Ne yapar?
`.büyük_harf()`	Bütün harfleri BÜYÜK yapar
`.küçük_harf()`	Bütün harfleri küçük yapar
`.baş_harf_büyük()`	Sadece ilk harfi büyütür
`.başlık_yap()`	Her kelimenin ilk harfini büyütür

Komut	Ne yapar?
`uzunluk(yazı)`	Kaç harf olduğunu sayar

Dikkat

Türkçedeki **i** harfini büyütürken bilgisayar bazen **İ** yerine **I** yazar. "deniz" yazısı **DENIZ** çıkar! Bu, bilgisayarın İngilizce kurallarıyla çalışmasından gelir. Bir hata değil; sadece bilmen gereken bir tuhaflık.

Mini Görev

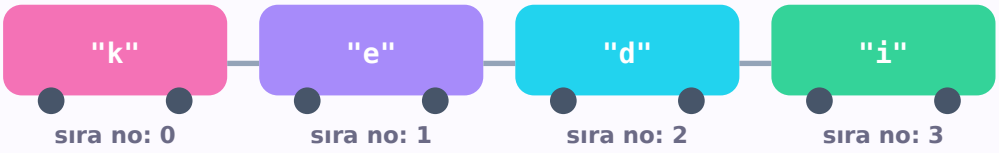
Kendi adını küçük harflerle bir kutuya koy. Sonra hem **büyük_harf()** hem de **baş_harf_büyük()** ile yazdır!

BÖLÜM 9

Harflere numara vermek

Her harfin bir sıra numarası vardır.

Bir yazıdaki harfler tren vagonları gibi dizilir. Ve bilgisayar saymaya **sıfırdan** başlar!



harf.trpy

```
ad = "kedi"
yazdır(ad[0])
yazdır(ad[3])
yazdır(ad[-1])
yazdır(uzunluk(ad))
```

>_ TERMİNAL

```
k
i
i
4
```

Köşeli parantez içindeki sayıya **sıra no** (indeks) denir. **-1** ise "sondan birinci" demektir.

Yazıları toplamak ve çoğaltmak

çoğalt.trpy

```
yazdır("Kodi" + " " + "robot")
yazdır("Ha" * 3)
yazdır("-" * 10)
```

>_ TERMİNAL

```
Kodi robot
HaHaHa
----------
```

Bir şey içinde var mı?

içinde.trpy

```
yazdır("ed" içinde "kedi")
yazdır("z" içinde "kedi")
```

>_ TERMİNAL

```
Doğru
Yanlış
```

Biliyor muydun?

içinde soruyu cevaplar: "Bu yazı, şu yazının içinde var mı?" Sonuç mantıksaldır; ama TürKod bunu ekranda da Türkçe gösterir: **Doğru** / **Yanlış**.
Kodunda ne yazıyorsan ekranda da onu görürsün. Hatırlaması çok kolay!

Bil bakalım

"Merhaba"[0] hangi harfi verir? Sıfırdan saydığını unutma!

BÖLÜM 10

Bilgisayar sana soru sorsun

Klavyeden cevap yazarsın, program onu kutuya koyar.

girdi_al komutu ekrana bir soru yazar ve senin cevabını bekler. Verdiğin cevap bir kutuya girer.

merhaba.trpy

```
1 isim = girdi_al("Adın ne? ")
2 yazdır(f"Merhaba {isim}!")
```

>_ TERMİNAL

```
Adın ne? Deniz
Merhaba Deniz!
```

Terminalde **sarı yazı** senin yazdığın cevaptır.

Sayı sormak istersen...

yaş.trpy

```
1 yaş = tamsayı(girdi_al("Kaç yaşındasın? "))
2 yazdır(f"Gelecek yıl {yaş + 1} yaşında olacaksın!")
```

>_ TERMİNAL

```
Kaç yaşındasın? 7
Gelecek yıl 8 yaşında olacaksın!
```

Biliyor muydun?

Klavyeden yazdığın her şey **metindir**. Hesap yapmak için önce **tamsayı( )** ile sayıya çevirmelisin. Yoksa 7 ile 1 toplanmaz, yan yana yapışıp "71" olur!

yapışma.trpy

```
1 a = girdi_al("Bir sayı yaz: ")
2 yazdır(a + "1")
```

>_ TERMİNAL

```
Bir sayı yaz: 7
71
```

İpucu

Virgüllü sayı sormak istersen **ondalıklı(girdi_al(...))** kullan.

BÖLÜM 10

Konuşan program: sohbet robotu

Öğrendiklerimizle küçük bir sohbet yapalım.

sohbet.trpy

```
ad = girdi_al("Merhaba! Adın ne? ")
renk_adı = girdi_al("En sevdiğin renk ne? ")
yaş = tamsayı(girdi_al("Kaç yaşındasın? "))

yazdır(f"Memnun oldum {ad}!")
yazdır(f"Ben de {renk_adı} rengini severim.")
yazdır(f"5 yıl sonra {yaş + 5} yaşında olacaksın.")
```

>_ TERMİNAL

```
Merhaba! Adın ne? Deniz
En sevdiğin renk ne? mavi
Kaç yaşındasın? 7
Memnun oldum Deniz!
Ben de mavi rengini severim.
5 yıl sonra 12 yaşında olacaksın.
```

Kod nasıl çalışıyor?

1. **Üç soru sorulur**
   Her **girdi_al** cevabı bir kutuya koyar: **ad**, **renk_adı**, **yaş**.

2. **Yaş sayıya çevrilir**
   tamsayı( ) sayesinde **yaş + 5** hesabı yapılabilir.

3. **f-yazılarla cevap verilir**
   Kutuların içindekiler cümlelerin arasına yerleşir.

! **Dikkat**

Yaş sorusuna harf yazarsan program **hata verir** çünkü harfleri sayıya çeviremez. Bu normal! Hata bölümünde bunu düzeltmeyi öğreneceğiz.

★ **Mini Görev**

Sohbet robotuna bir soru daha ekle: "En sevdiğin hayvan ne?" Cevabı da bir cümlede kullan!

BÖLÜM 11

eğer: karar vermeyi öğren

Bilgisayar da bazen "olursa şunu yap" der.

Sabah dışarı çıkarken düşünürsün: "Eğer yağmur yağıyorsa şemsiye alırım." Bilgisayarlar da böyle karar verir. Bunun için **eğer** kelimesini kullanırız.

eğer.trpy

```
yaş = 8
eğer yaş >= 7:
    yazdır("Okula gidebilirsin!")
```

>_ TERMİNAL

```
Okula gidebilirsin!
```

Kodun sırrı: iki nokta ve içeri girme

1. **İki nokta (:)**
   eğer satırının sonunda iki nokta vardır. Şöyle der: "bu doğruysa aşağıdakini yap".

2. **Girinti (4 boşluk)**
   Yapılacak iş içeriden başlar. Satırın önüne 4 boşluk (ya da Tab) koyarız. Buna **girinti** denir.

3. **Şart yanlışsa atlanır**
   İçerideki satırlar hiç çalışmaz. Bilgisayar onları görmezden gelir.

> ! **Dikkat**
> Girintiyi unutursan bilgisayar hangi satırların **eğer**'e ait olduğunu anlayamaz ve hata verir. İçeri girmeyi asla unutma!

BÖLÜM 11

değilse: ya olmazsa?

İki yoldan birini seç.

eğer şartı doğru değilse ne olur? Bunun için **değilse** yazarız. Böylece bilgisayarın iki yolu olur.



değilse.trpy

```
1 hava = "yağmurlu"
2 eğer hava == "güneşli":
3     yazdır("Parka gidelim!")
4 değilse:
5     yazdır("Evde oyun oynayalım!")
```

>_ TERMİNAL

```
Evde oyun oynayalım!
```

Burada **hava** "yağmurlu". "güneşli" ile aynı olmadığı için bilgisayar **değilse** yolunu seçti.

★ **Mini Görev**

hava kutusunun içine "güneşli" yaz ve çalıştır. Hangi yol seçildi? Sonra sen bir hava durumu daha dene!

? **Bil bakalım**

değilse satırının sonunda da iki nokta (:) var mı? Yukarıya bak ve kontrol et!

BÖLÜM 11

değilse_eğer: çok yollu seçim

Üç ya da daha fazla ihtimal olduğunda.

Bazen ikiden fazla ihtimal vardır. Bir yarışmada puan 90 ya da üstü ise altın, 60 ya da üstü ise gümüş madalya verilir. Bunu **değilse_eğer** ile yazarız.



madalya.trpy

```
puan = 75
eğer puan >= 90:
    yazdır("Harika! Altın madalya!")
değilse_eğer puan >= 60:
    yazdır("Güzel! Gümüş madalya!")
değilse:
    yazdır("Bir daha dene, olur!")
```

>_ TERMİNAL

```
Güzel! Gümüş madalya!
```

Bilgisayar yukarıdan aşağıya sırayla sorar. Puan 75: "90 ya da üstü mü? Hayır. 60 ya da üstü mü? Evet!" Ve gümüş madalyayı verir. Doğru bir şart bulunca **kalan şartlara bakmaz**.

İpucu

Sıra: önce **eğer**, sonra istediğin kadar **değilse_eğer**, en sonda da (istersen) bir tane **değilse**.

Mini Görev

Puanı 95 yap, sonra 40 yap ve çalıştır. Her seferinde hangi madalya çıktı?

BÖLÜM 11

Karşılaştırma işaretleri

Bilgisayar iki şeyi karşılaştırıp Doğru ya da Yanlış der.

İşaret	Sorduğu soru	Örnek	Cevap
==	eşit mi?	5 == 5	Doğru
!=	farklı mı?	5 != 5	Yanlış
>	büyük mü?	8 > 3	Doğru
<	küçük mü?	8 < 3	Yanlış
>=	büyük ya da eşit mi?	7 >= 7	Doğru
<=	küçük ya da eşit mi?	9 <= 4	Yanlış

Dikkat

Çok önemli! Bir eşittir (**=**) kutuya bir şey **koyar**. İki eşittir (**==**) ise **sorar**: "bunlar aynı mı?" Karıştırırsan program şaşırır.

iki-eşittir.trpy

```
yaş = 7
eğer yaş == 7:
    yazdır("Yedi yaşındasın!")
```

>_ TERMİNAL

```
Yedi yaşındasın!
```

yaş = 7 kutuya 7 koydu. **yaş == 7** ise "yaş 7 mi?" diye sordu. Cevap Doğru olduğu için mesaj yazıldı.

İpucu

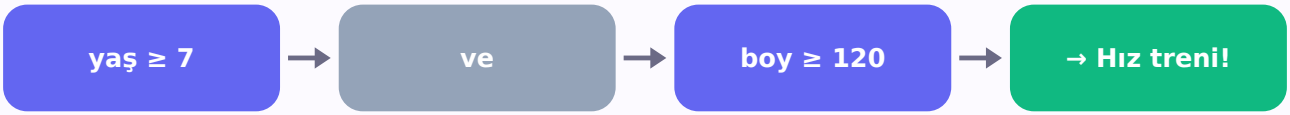
Karşılaştırma işaretlerini **eğer** ve **döngü** satırlarında çok kullanacaksın.

BÖLÜM 11

ve, veya, değil

Birden fazla şartı birlikte sorabilirsin.

Hız trenine binmek için yaşın 7 veya üstü olmalı, **hem de** boyun 120 veya üstü olmalı. İki şart aynı anda gerekiyorsa **ve** kullanırız.



hız-treni.trpy

```
yaş = 8
boy = 130
eğer yaş >= 7 ve boy >= 120:
    yazdır("Hız treni seni bekliyor!")
```

>_ TERMİNAL

```
Hız treni seni bekliyor!
```

Kelime	Ne demek?
ve	İki şart da doğru olmalı
veya	Şartlardan biri doğru olsa yeter
değil	Cevabı tersine çevirir

veya-değil.trpy

```
güneş = Yanlış
yağmur = Doğru
eğer güneş veya yağmur:
    yazdır("Dışarıda hava değişik!")
eğer değil güneş:
    yazdır("Güneş yok, ceket giy!")
```

>_ TERMİNAL

```
Dışarıda hava değişik!
Güneş yok, ceket giy!
```

Güneş Yanlış, yağmur Doğru. **veya** için biri doğru olsa yeter. **değil güneş** ise Yanlış'ı Doğru'ya çevirir.

★ **Mini Görev**

Kendi yaş ve boy kutularını yap. **ve** ile "hem 6 yaşından büyük hem 110 boydan uzun mu?" diye sor.

BÖLÜM 12

döngü: tekrar tekrar yapmak

Aynı işi 100 kere yazmaktan kurtul!

Bir çocuğa "5 kere zıpla" dersin. Zıpla, zıpla, zıpla... diye 5 kere söylemezsin. Bilgisayara da böyle söyleriz: bu işe **döngü** denir.

zıpla.trpy

```
1 döngü i aralık(5):
2     yazdır("Zıpla!")
```

>_ TERMİNAL

```
Zıpla!
Zıpla!
Zıpla!
Zıpla!
Zıpla!
```

`döngü i aralık(5):` demek: "i adlı sayacı 5 kere say ve içerideki işi her seferinde yap". İçerideki satırlar yine 4 boşluk içeriden başlar.

sayaç.trpy

```
1 döngü i aralık(3):
2     yazdır(i)
```

>_ TERMİNAL

```
0
1
2
```

Biliyor muydun?

Sayaç 0'dan başlar! Bilgisayarlar saymaya sıfırdan başlar. Garip ama doğru; ileride listelerde de aynı kuralı göreceksin.

Mini Görev

"Sevgiler!" yazısını 8 kere yazdıran bir program yap.

BÖLÜM 12

aralık: sayı yolculuğu

Nereden başlayıp nerede duracağını sen seçersin.

`aralık` bir sayı yolu çizer. Üç şekilde yazabilirsin:

Yazım	Başlangıç	Bitiş (dahil değil)	Verdiği sayılar
`aralık(5)`	0	5	0, 1, 2, 3, 4
`aralık(1, 6)`	1	6	1, 2, 3, 4, 5
`aralık(2, 11, 2)`	2	11	2, 4, 6, 8, 10 (ikişer atlar)

Dikkat

Bitiş sayısı **dahil değildir**. aralık(1, 6) sayıları 1, 2, 3, 4, 5 verir. Altıya ulaşmadan durur!

Geri sayım roketi

roket.trpy

```
1 döngü sayı aralık(5, 0, -1):
2     yazdır(sayı)
3 yazdır("Fırla!")
```

```
>_ TERMİNAL
5
4
3
2
1
Fırla!
```

Adım **-1** olursa geriye doğru sayarız!

Mini Görev

aralık(1, 21) ile 1'den 20'ye kadar sayan bir döngü yaz.

BÖLÜM 12

Şart sürdükçe tekrarla

Kaç kere yapacağını bilmiyorsan bu döngü tam sana göre!

Bazen "enerjin bitene kadar oyna" dersin. Kaç kere olacağı belli değil, şart belli. Bunun için **`döngü şart:`** yazarız. Şart Doğru olduğu sürece döngü döner.

enerji.trpy

```
1 enerji = 3
2 döngü enerji > 0:
3     yazdır(f"Enerji: {enerji}")
4     enerji = enerji - 1
5 yazdır("Uyku zamanı!")
```

>_ TERMİNAL

```
Enerji: 3
Enerji: 2
Enerji: 1
Uyku zamanı!
```

`enerji = enerji - 1` satırı çok önemli: enerjiyi her turda azaltıyor. Yoksa şart hiç Yanlış olmaz ve döngü **sonsuza kadar** döner!

Dikkat

Program bitmiyorsa panik yok! Araç çubuğundaki kırmızı **Durdur** düğmesine basarak durdurabilirsin.

Mini Görev

Enerjiyi 5 yap ve çalıştır. Kaç satır yazıldı?

BÖLÜM 12

kır ve devam_et

Döngüyü durdur ya da bir turu atla.

kır döngüden tamamen çıkar. **devam_et** yalnızca o turu atlar ve döngü kaldığı yerden sürer.

kır: dur ve çık

kır.trpy

```
döngü sayı aralık(1, 10):
    eğer sayı == 4:
        kır
    yazdır(sayı)
```

>_ TERMİNAL

```
1
2
3
```

Sayı 4 olunca **kır** çalıştı ve döngü bitti. 4 ve sonrası yazılmadı.

devam_et: bu turu atla

devam.trpy

```
döngü sayı aralık(1, 6):
    eğer sayı == 3:
        devam_et
    yazdır(sayı)
```

>_ TERMİNAL

```
1
2
4
5
```

Sayı 3 olunca tur atlandı ama döngü 4 ve 5 ile devam etti.

Bil bakalım

Yukarıdaki iki çıktıda hangi sayılar eksik? **kır** ile **devam_et** arasındaki farkı kendi kelimelerinle anlat.

BÖLÜM 12

İç içe döngüler

Bir döngünün içine başka bir döngü koyabilirsin.

Bir çarpım tablosunu değil, **bütün** çarpım tablosunu istersek döngünün içine başka bir döngü koyarız.

içiçe.trpy

```
1 döngü satır aralık(1, 4):
2     döngü sütun aralık(1, 4):
3         yazdır(f"{satır} x {sütun} = {satır * sütun}")
4     yazdır("---")
```

>_ TERMİNAL

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
---
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
---
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
---
```

Dış döngü **satır** her döndüğünde, iç döngü **sütun** baştan sona kadar tamamen döner. Bu yüzden toplam 3 x 3 = 9 satır çıktı olur.

★ **Mini Görev**

İç içe döngüyle 5x5'lik bir yıldız (*) karesi yazdırmayı dene: her satırda 5 tane * yan yana.

BÖLÜM 13

Listeler: vagonlu tren

Birçok şeyi tek kutuda tut!

Bir kutuda tek şey yerine birçok şey tutmak istersen **liste** kullanırız. Liste bir tren gibidir: her vagonda bir şey var. Köşeli parantez **[ ]** içine yazarız, aralarına virgül koyarız.



meyveler.trpy

```
meyveler = ["elma", "armut", "kiraz"]
yazdır(meyveler)
yazdır(uzunluk(meyveler))
```

>_ TERMİNAL

```
['elma', 'armut', 'kiraz']
3
```

`uzunluk(meyveler)` trende kaç vagon olduğunu söyler: 3.

Vagon içine bakmak: sıra no

vagon-oku.trpy

```
meyveler = ["elma", "armut", "kiraz"]
yazdır(meyveler[0])
yazdır(meyveler[2])
yazdır(meyveler[-1])
```

>_ TERMİNAL

```
elma
kiraz
kiraz
```

Biliyor muydun?
Listede de saymak **0**'dan başlar. İlk vagon 0, ikinci vagon 1'dir. **-1** ise en son vagondur.

Yeni vagon eklemek

vagon-ekle.trpy

```
meyveler = ["elma", "armut"]
meyveler.sona_ekle("kiraz")
yazdır(meyveler)
```

>_ TERMİNAL

```
['elma', 'armut', 'kiraz']
```

BÖLÜM 13

Listeyi düzenlemek

Sırala, ters çevir, topla!

sıralama.trpy

```
1 meyveler = ["elma", "armut"]
2 meyveler.sona_ekle("kiraz")
3 yazdır(meyveler)
4 meyveler.sırala()
5 yazdır(meyveler)
6 meyveler.tersine_çevir()
7 yazdır(meyveler)
```

>_ TERMİNAL

```
['elma', 'armut', 'kiraz']
['armut', 'elma', 'kiraz']
['kiraz', 'elma', 'armut']
```

`sırala()` alfabe sırasına dizer, **`tersine_çevir()`** ise sırayı tersine çevirir.

Sayı listeleri

notlar.trpy

```
1 notlar = [70, 85, 100]
2 yazdır(toplam(notlar))
3 yazdır(en_büyük(notlar))
4 yazdır(en_küçük(notlar))
```

>_ TERMİNAL

```
255
100
70
```

Komut	Ne yapar?	Komut	Ne yapar?
`sona_ekle(x)`	En sona x ekler	`çıkar()`	Son vagonu çıkarır
`kaldır(x)`	x değerini siler	`say(x)`	x kaç kere var?
`temizle()`	Listeyi boşaltır	`toplam(liste)`	Sayıları toplar
`en_büyük(liste)`	En büyük sayı	`en_küçük(liste)`	En küçük sayı

Bil bakalım

Notlar listesi [70, 85, 100] idi. Hangi komut 100'ü, hangisi 70'i bulur? Yukarıya bakmadan söyle!

BÖLÜM 13

Liste içinde dolaşmak

Her vagonu tek tek ziyaret et.

Listedeki her şeyi tek tek işlemek istersen döngüyle birlikte **içinde** kullanırız.

dolaş.trpy

```
1 meyveler = ["elma", "armut", "kiraz"]
2 döngü meyve içinde meyveler:
3     yazdır(f"Ben {meyve} seviyorum!")
```

>_ TERMİNAL

```
Ben elma seviyorum!
Ben armut seviyorum!
Ben kiraz seviyorum!
```

Her turda **meyve** kutusuna listenin bir sonraki elemanı gelir. Sıra no ile uğraşmana gerek kalmaz!

Bir şey listede var mı?

liste-içinde.trpy

```
1 meyveler = ["elma", "armut", "kiraz"]
2 yazdır("armut" içinde meyveler)
3 yazdır("muz" içinde meyveler)
```

>_ TERMİNAL

```
Doğru
Yanlış
```

Dikkat

Çıktıda **Doğru** / **Yanlış** gördün mü? Kodunda yazdığın kelimeler ekranda da aynen görünür: TürKod mantıksal sonuçları hep Türkçe yazar.

Mini Görev

Sevdiğin 4 oyuncağın listesini yap, sonuna bir tane daha ekle ve döngüyle hepsini tek tek yazdır.

BÖLÜM 14

Fonksiyon: kendi komutunu yap

Bir kere tarif yaz, istediğin kadar kullan.

Her gün dişini fırçalamak için aynı adımları tekrar tekrar anlatmazsın, değil mi? "Diş fırçala" dersin, gerisi kendiliğinden bilinir. Kod da böyle: sık kullandığımız işlemlere kendi seçtiğimiz bir adı veririz. Buna **fonksiyon** denir.

selamla.trpy

```
1 fonksiyon selamla(isim):
2     yazdır(f"Merhaba {isim}!")
3
4 selamla("Deniz")
5 selamla("Ada")
```

>_ TERMİNAL

```
Merhaba Deniz!
Merhaba Ada!
```

Parçalarına bakalım

Parça	Ne demek?
`fonksiyon`	Yeni bir komut yaratıyoruz demek
`selamla`	Komutun adı. İstediğin adı verebilirsin
`(isim)`	Komuta verilen bilgi. Buna parametre denir
içerideki satırlar	Komut çağrılınca yapılacak işler (girintili)
`selamla("Deniz")`	Komutu çağırıyoruz! Şimdi çalışır

> **Dikkat**
> Fonksiyonu **tanımlamak** onu çalıştırmaz. Çalışması için adını yazıp **çağırmalısın**.

BÖLÜM 14

döndür: makineden sonuç al

Fonksiyon bir sonuç geri verebilir.

Bazı fonksiyonlar ekrana yazmaz, bir sonuç **geri verir**. Bir makine gibi düşün: içine bir şey atarsın, makine çalışır ve sonucu dışarı verir. Sonucu geri vermek için **döndür** kullanırız.



kare.trpy

```
1 fonksiyon kare(x):
2     döndür x * x
3
4 yazdır(kare(4))
5 yazdır(kare(7))
```

```
>_ TERMİNAL
16
49
```

döndür x * x demek: "x ile x'i çarp ve sonucu geri ver".

İki bilgi alan makine

topla-iki.trpy

```
1 fonksiyon topla(a, b):
2     döndür a + b
3
4 sonuç = topla(3, 4)
5 yazdır(f"Sonuç: {sonuç}")
```

```
>_ TERMİNAL
Sonuç: 7
```

Mini Görev

Bir sayının 2 katını döndüren **iki_katı** adında bir fonksiyon yaz. Sonra **yazdır(iki_katı(21))** ile dene: 42 çıkmalı!

BÖLÜM 14

Varsayılan değer ve birden çok bilgi

Fonksiyonlar esnektir.

Bir parametreye **önceden bir değer** verebilirsin. Kullanan kişi o değeri değiştirmezse, varsayılan değer kullanılır.

varsayılan.trpy

```
1 fonksiyon selamla(isim, mesaj="Merhaba"):
2     yazdır(f"{mesaj} {isim}!")
3
4 selamla("Deniz")
5 selamla("Ada", "Selam")
```

>_ TERMİNAL

```
Merhaba Deniz!
Selam Ada!
```

İlk çağrıda **mesaj** verilmedi, o yüzden varsayılan "Merhaba" kullanıldı. İkincisinde biz "Selam" dedik, o kullanıldı.

Biliyor muydun?

Kodun tekrarını azaltmak programcılığın en önemli kurallarından biridir. Aynı işi iki kere yazıyorsan, muhtemelen bir fonksiyon yapman gerekiyordur!

BÖLÜM 15

Hata avcısı olalım

Hata yapmak çok normal. Önemli olan bulup düzeltmek!

Koddaki bir hataya **bug** (böcek) denir. En iyi programcılar bile her gün böcek avlar! Bilgisayar bir hata bulursa sana mesaj yazar. Mesajı dikkatle oku, ne yanlış gitmiş söyler.

1. Kutunun adı yanlış

hata1.trpy

```
1 yazdır(isim)
```

```
>_ TERMİNAL
[HATA]: Tanımsız İsim Hatası (Satır Numarası: 1)
Açıklama: 'isim' adında bir değişken/fonksiyon tanımlı değil!
```

Hiç yapmadığın bir kutuyu yazdırmaya çalıştın. Önce `'isim = "Deniz"` yazmalısın.

2. Parantez açık kaldı

hata2.trpy

```
1 yazdır("Merhaba"
```

```
>_ TERMİNAL
[Hata] Satır 1, sütun 6: '(' açıldı ama kapatılmadı
```

Parantezi açtın ama kapatmayı unuttun. Sonuna **)** koy.

3. Sıfıra bölmek

hata3.trpy

```
1 yazdır(10 / 0)
```

```
>_ TERMİNAL
[HATA]: Sıfıra Bölme Hatası (Satır Numarası: 1)
Açıklama: bir sayı 0'a bölünemez!
```

Sıfıra bölmek imkânsızdır! Bölen sayıyı sıfırdan farklı yap.

BÖLÜM 15

Daha çok hata türü

Ne kadar çok hata görürsen o kadar çabuk tanırsın.

4. Yazı ile sayıyı toplamak

hata4.trpy

```
1 yaş = 7
2 yazdır("Yaşım: " + yaş)
```

>_ TERMİNAL

```
[HATA]: Veri Tipi Hatası (Satır Numarası: 2)
Açıklama: metin ile sayı gibi farklı türler + ile birleştirilemez!
```

Yazıyı ve sayıyı doğrudan **+** ile birleştiremezsin. Önce **metin(yaş)** ile çevir ya da f-yazı kullan.

5. Girinti unutulmuş

hata5.trpy

```
1 eğer Doğru:
2 yazdır("İçerde miyim?")
```

>_ TERMİNAL

```
[Hata] Satır 2, sütun 1: girintili blok bekleniyor
```

eğer satırından sonraki satır **4 boşluk** içeri girmeli.

6. Listede olmayan bir vagona bakmak

hata6.trpy

```
1 liste = [1, 2, 3]
2 yazdır(liste[10])
```

>_ TERMİNAL

```
[HATA]: Liste İndeks Hatası (Satır Numarası: 2)
Açıklama: listenin sınırları dışında bir elemana ulaşmaya çalıştınız!
```

Listede sadece 3 vagon var (0, 1, 2 numaralı). 10 numaralı vagon yok!

★ **İpucu**

Programdaki hata mesajının sözcükleri biraz farklı olabilir. Ama fikri aynıdır: mesajı oku, hangi satır olduğuna bak.

i **Biliyor muydun?**

Editörün altındaki **Sözdizimi** yazısı ve **AI Asistan** hata ararken yardımcı olabilir. Ama önce kendin düşün!

BÖLÜM 16

Bonus: Kaplumbağa ile çizim

Ekrana kod yazmak yerine, kod yazıp resim çizelim!

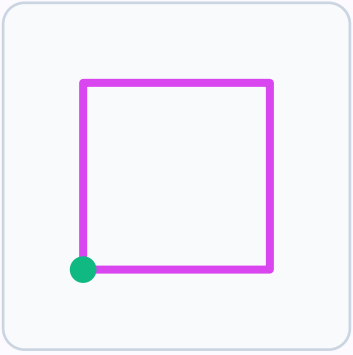
TürKod'da **kaplumbaga** adında sihirli bir kütüphane var. Ekranda küçük bir ok (kaplumbağamız) yürür, dönerken arkasında iz bırakır.

kare-çiz.trpy

```
içe_aktar kaplumbaga

döngü i aralık(4):
    kaplumbaga.ileri(100)
    kaplumbaga.sağa_dön(90)

kaplumbaga.cikista_bekle()
```



Komut	Ne yapar?
`ileri(100)`	100 adım ileri gider
`geri(50)`	50 adım geri gider
`sağa_dön(90)`	90 derece sağa döner
`sola_dön(90)`	90 derece sola döner
`kalem_rengi("kırmızı")`	Çizgi rengini değiştirir
`kalem_kaldır()`	İz bırakmadan hareket eder
`kalem_indir()`	Yeniden iz bırakmaya başlar

Döngüyle birleştirince kaplumbağa harika şekiller çizebilir. Kare için 4 kere "ileri git, 90 derece dön" yeterlidir - matematikte 4 x 90 = 360 derece, yani tam bir tur!

★ **Mini Görev**

Döngüyü 5 kere çalıştırıp her seferinde 72 derece döndürerek bir beşgen çizmeyi dene (5 x 72 = 360).

i **Biliyor muydun?**

Bu bölümdeki kaplumbağa kodları, sitedeki Canlı Demo'da değil, yalnızca kurulu **TürKod IDE**'de bir pencere açarak çalışır.

BÖLÜM 17

Mini proje 1: Yaş kahramanı

Öğrendiklerimizi birleştirelim!

Bu programda bilgisayar yaşını soruyor ve yaşına göre bir unvan veriyor. **girdi_al**, **tamsayı** ve **değilse_eğer** bir arada!

yaş-kahramanı.trpy

```
yaş = tamsayı(girdi_al("Kaç yaşındasın? "))
eğer yaş < 3:
    yazdır("Bebek kod kahramanı!")
değilse_eğer yaş < 13:
    yazdır("Çocuk kod kahramanı!")
değilse:
    yazdır("Büyük kod kahramanı!")
```

>_ TERMİNAL

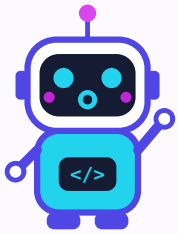
```
Kaç yaşındasın? 7
Çocuk kod kahramanı!
```

Kod nasıl çalışıyor?

1. **Sor ve çevir**
   Bilgisayar yaşını sorar, tamsayı yapıp **yaş** kutusuna koyar.

2. **Sırayla kontrol et**
   Yaş 3'ten küçükse ilk mesaj, 13'ten küçükse ikinci mesaj yazılır.

3. **Hiçbiri değilse**
   değilse kısmı çalışır.



Yaşını 2, 10 ve 40 yaparak dene. Her seferinde hangi mesaj çıkıyor?

★ **Mini Görev**

Dördüncü bir unvan ekle: 60'tan büyük olanlara "Bilge kod kahramanı!" desin.

BÖLÜM 17

Mini proje 2: Alışveriş listesi

Listeler ve yazılar bir arada!

Alışveriş listesine iki şey daha ekleyip alfabe sırasına dizelim. Sonra kaç şey aldığımızı sayalım.

alışveriş.trpy

```
alışveriş = ["süt", "ekmek"]
alışveriş.sona_ekle("elma")
alışveriş.sona_ekle("çilek")
alışveriş.sırala()
yazdır(f"Alacaklarım: {alışveriş}")
yazdır(f"Toplam {uzunluk(alışveriş)} şey")
```

>_ TERMİNAL

```
Alacaklarım: ['ekmek', 'elma', 'süt', 'çilek']
Toplam 4 şey
```

Neler oluyor?

1. **Liste iki şeyle başlıyor**
   süt ve ekmek.

2. **sona_ekle ile ekleniyor**
   elma ve çilek en sona eklendi.

3. **sırala() alfabeye diziyor**
   ekmek, elma, süt, çilek.

4. **uzunluk() sayıyor**
   Vagon sayısını, yani 4'ü verdi.

Biliyor muydun?
Bilgisayar **ç**, **ş**, **ü** gibi Türkçe harfleri z'den sonra dizer. O yüzden çilek sona geçti!

Mini Görev
Kendi alışveriş ya da oyuncak listeni yap, iki şey ekle, sırala ve say!

BÖLÜM 17

Mini proje 3: Çarpım tablosu

Döngü ve hesap makinesi birlikte!

İstediğin sayının çarpım tablosunu saniyeler içinde yazdıran bir program yapalım. Elle yazmak uzun sürerdi, ama döngü bunu bizim için yapıyor.

çarpım-tablosu.trpy

```
sayı = tamsayı(girdi_al("Hangi sayının tablosu? "))
döngü i aralık(1, 11):
    yazdır(f"{sayı} x {i} = {sayı * i}")
```

>_ TERMİNAL

```
Hangi sayının tablosu? 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
4 x 10 = 40
```

`sayı * i` önce hesaplanır, sonra f-yazının içine yerleşir. Bu yüzden çarpım işlemi doğru sırada olur.

Biliyor muydun?

Sen de yapabilirsin! **aralık(1, 11)** yerine **aralık(1, 21)** yazarsan 20'ye kadar tablo çıkar. Bilgisayar hiç yorulmaz!

Mini Görev

Kendi projeni tasarla: Doğum günü yaş hesaplayıcısı, Hayvan listesi ya da Geri sayım roketi. Hangisini yapacaksın?

BÖLÜM 17

Mini proje 4: Tahmin oyunu

Fonksiyonlar, döngü ve karar bir arada!

Bilgisayar aklından 1 ile 10 arası bir sayı tutuyor. Sen tahmin ediyorsun, bilgisayar "küçük" ya da "büyük" diyor.

tahmin.trpy

```
1 gizli_sayı = 6
2 tahmin = tamsayı(girdi_al("1-10 arası bir sayı tahmin et: "))
3
4 eğer tahmin == gizli_sayı:
5     yazdır("Bildin! Tebrikler!")
6 değilse_eğer tahmin < gizli_sayı:
7     yazdır("Daha büyük bir sayı dene.")
8 değilse:
9     yazdır("Daha küçük bir sayı dene.")
```

>_ TERMİNAL

```
1-10 arası bir sayı tahmin et: 4
Daha büyük bir sayı dene.
```

Mini Görev

Gizli sayıyı rastgele yapmak ister misin? `içe_aktar rastgele` yazdıktan sonra `rastgele.rastgele_tamsayı(1, 10)` komutunu kullanabilirsin. Bunu AI Asistan'a da sorabilirsin!

BÖLÜM 18

Kod kahramanı bilmeceleri

Kodu oku, bilgisayarın ne yazacağını tahmin et. Sonra cevaplara bak!

1 Ne yazılır?

```
x = 3
x = x + 2
yazdır(x)
```

2 "Hop!" kaç kere yazılır?

```
döngü i aralık(3):
    yazdır("Hop!")
```

3 Ne yazılır?

```
yazdır(10 % 3)
```

4 Hangi harf yazılır?

```
eğer 5 > 8:
    yazdır("A")
değilse:
    yazdır("B")
```

5 Ne yazılır?

```
yazdır(uzunluk("elma"))
```

6 Hangi sayılar yazılır?

```
döngü i aralık(1, 4):
    yazdır(i)
```

No	Sorusu
7	x = 4; eğer x > 3 ve x < 10: yazdır("Evet") değilse: yazdır("Hayır") — Evet mi Hayır mı?
8	ad = "ada"; yazdır(ad.büyük_harf()) — Ne yazılır?
9	= ile == arasındaki fark nedir?
10	Kendi komutunu yapmak için hangi kelimeyi kullanırız?

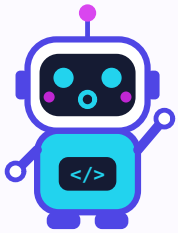
BÖLÜM 18

Bilmece cevapları

Kaç tanesini bildin? Kendini puanla!

No	Cevap	Açıklama
1	**5**	x önce 3, sonra 3 + 2 = 5 oldu.
2	**3 kere**	aralık(3) üç tur döner.
3	**1**	10 içinde 3 üç kere var (9 eder). Geriye 1 kalır.
4	**B**	5 > 8 Yanlış olduğu için değilse çalışır.
5	**4**	"elma" 4 harflidir.
6	**1, 2, 3**	Bitiş sayısı (4) dahil değil.
7	**Evet**	4, hem 3'ten büyük hem 10'dan küçük.
8	**ADA**	büyük_harf() bütün harfleri büyütür.
9	**= koyar, == sorar**	Tek eşittir kutuya değer koyar, çift eşittir eşit mi diye sorar.
10	**fonksiyon**	fonksiyon adı(...): ile yeni bir komut yaparız.

Doğru sayın	Rütbe
0-5 doğru	Kod Kaşifi
6-8 doğru	Kod Kahramanı
9-10 doğru	Kod Ustası



Kaç tanesini bildin? Ne olursa olsun, denemiş olman bile seni bir kahraman yapar!

BÖLÜM 19

Kodi'nin sözlüğü

Bu kitapta öğrendiğimiz sihirli kelimeler.

yazdır
Ekrana yazı yazar

girdi_al
Klavyeden cevap ister

eğer
Şart doğruysa çalışır

değilse
Şart yanlışsa çalışır

değilse_eğer
Başka bir şartı dener

döngü
Aynı işi tekrar eder

aralık
Sayı yolu oluşturur

içinde
Bir liste ya da yazı içinde dolaşır

kır
Döngüden tamamen çıkar

devam_et
Turu atlar, devam eder

fonksiyon
Yeni komut yapar

döndür
Sonucu geri verir

ve
İki şart da doğru olmalı

veya
Biri doğruysa yeter

değil
Doğruyu yanlışa çevirir

Doğru
Mantıksal değer: evet

Yanlış
Mantıksal değer: hayır

tip
Türü söyler

uzunluk
Kaç eleman ya da harf var?

tamsayı
Metni sayıya çevirir

ondalıklı
Metni virgüllü sayıya çevirir

metin
Bir değeri yazıya çevirir

toplam
Listeyi toplar

en_büyük / en_küçük
Listenin uç değerlerini bulur

sona_ekle
Listeye eleman ekler

sırala
Listeyi sıraya dizer

tersine_çevir
Listeyi ters çevirir

büyük_harf / küçük_harf
Harfleri büyütür / küçültür

baş_harf_büyük
İlk harfi büyütür

yuvarla
Sayıyı yuvarlar

mutlak
Eksi işaretini siler

BÖLÜM 19

Hızlı kopya kâğıdı

Unuttuğunda buraya bak!

Yazdır ve sor

```
yazdır("Merhaba")
isim = girdi_al("Adın? ")
```

Karar

```
eğer x > 5:
    yazdır("büyük")
değilse:
    yazdır("küçük")
```

Döngü

```
döngü i aralık(5):
    yazdır(i)
döngü x < 10:
    x = x + 1
```

Liste

```
liste = [1, 2, 3]
liste.sona_ekle(4)
yazdır(uzunluk(liste))
```

Fonksiyon

```
fonksiyon kare(x):
    döndür x * x
yazdır(kare(5))
```

İşaretler	Anlamı
+ - * / // % **	toplama, çıkarma, çarpma, bölme, tam bölme, kalan, kuvvet
== != > < >= <=	eşit, farklı, büyük, küçük, büyük-eşit, küçük-eşit
ve veya değil	mantıksal bağlaçlar

Biliyor muydun?
Editörde komutlar pembe, yazılar yeşil görünür. Renkler sana doğru yolda olup olmadığını söyler!

BÖLÜM 20

Büyükler için not

Bu kitabı çocuğunuzla birlikte nasıl kullanabilirsiniz?

Kitap nasıl kurgulandı?

Her sayfa tek bir fikir anlatır: önce benzetme, sonra kısa kod, sonra çıktı, en sonda mini görev. Çocuğunuz aynı anda yalnızca bir kavramla uğraşır.

Önerilen tempo

Günde 1-2 bölüm (yaklaşık 15-25 dakika) ile kitap iki-üç haftada biter. Her mini görevi çocuğunuzun kendi klavyesiyle denemesine izin verin. Hata yapması öğrenmenin parçasıdır.

Nasıl destek olabilirsiniz?

Kodu okuyup "sence ne yazacak?" diye sorun. Sonra çalıştırın. Tahmin ile sonucu karşılaştırmak, kavramı ezberden çok daha iyi oturtur.

Kurulum ve güvenlik

TürKod'u yalnızca resmî siteden ve büyüklerin gözetiminde indirin. Windows bir uyarı gösterirse devam etmeden önce bağlantının resmî olduğundan emin olun.

AI Asistan

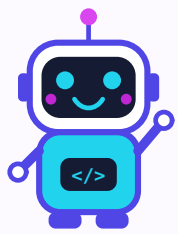
Yerleşik yapay zekâ asistanı bazen yanılabilir. Çocuğunuz kullanırken ona eşlik edin ve verilen cevabı birlikte kontrol edin.

Örneklerin doğruluğu

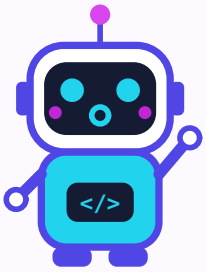
Bu kitaptaki kodların çıktıları, TürKod IDE üzerinde gerçekten çalıştırılarak elde edildi ve bu düzeltilmiş baskıda tekrar doğrulandı. Programın bazı ayrıntıları ve hata mesajlarının sözcükleri sürümden sürüme farklılık gösterebilir.

Bağlantılar

Site: yusufx-sys.github.io/turkod-site
Program: github.com/YusufX-sys/turkod-ide
Site kaynağı: github.com/YusufX-sys/turkod-site



Birlikte harika bir yolculuk olacak! Takılırsanız kitabın sözlük ve kopya kâğıdı sayfalarına bakın.



KOD KAHRAMANI SERTİFİKASI

Bu sertifika şu kahramana verilmiştir:

_____

(adını buraya yaz)

TürKod ile Kodlama Macerası kitabını tamamladığı; değişkenleri, karar vermeyi, döngüleri, listeleri ve fonksiyonları öğrendiği ve ilk programlarını yazdığı için **KOD KAHRAMANI** unvanına layık görülmüştür.

_____

Tarih

Kodi

İmza